

STEM Agent: A Self-Adapting, Tool-Enabled, Extensible Architecture for Multi-Protocol AI Agent Systems in Production

Anonymous ACL submission

Abstract

Production AI agent systems increasingly suffer from *architectural lock-in*: early commitment to a single interaction protocol, a fixed tool-integration path, and a static user model makes deployed agents hard to re-target when requirements, client surfaces, or emerging standards shift. We describe **STEM Agent** (Self-adapting, Tool-enabled, Extensible, Multi-agent), an architecture designed for enterprise deployment that must simultaneously serve human end-users, other agents, web UIs, and commerce backends. STEM Agent unifies five interoperability protocols—A2A, AG-UI, A2UI, UCP, and AP2—behind a single gateway, isolates domain knowledge behind the Model Context Protocol (MCP), and continuously adapts per-user behavior through a Caller Profiler that learns preferences across 21 behavioral dimensions. Recurring interaction patterns *crystallize* into reusable skills through a maturation lifecycle. We report operational lessons from six months of internal deployment serving approximately 200 weekly active users across four production use cases (customer support, internal search, code assistance, documentation QA), and we additionally ship five industry-aligned persona templates—SRE, finance, AI research, healthcare, and legal—including regulated personas that enforce HIPAA-style PHI guardrails and privileged-communication boundaries through a typed forbiddenTopics contract. We discuss scalability patterns (horizontal scaling via externalized state, Kubernetes HPA with readiness/liveness probes), an *opt-in* enterprise hardening surface (RBAC middleware, SIEM-ready audit stream, strict CSP/HSTS headers, Prometheus /metrics, default PII redaction), pluggable IAM (JWT/OAuth2/SAML/API-Key), and production deployment artifacts (multi-stage non-root Dockerfile, Kubernetes manifests) shipped with the system. All hardening controls default off so the quickstart path remains untouched; enabling them is a per-flag

decision that mirrors the concerns deployment teams raise during security review.

1 Introduction

Teams deploying LLM-based agents inside a large enterprise quickly discover that the hard problems are not which reasoning prompt to use, but how the agent connects to the rest of the business: which protocol it speaks to other agents, which payment or commerce standard it must satisfy, how it streams partial progress to a web UI, and how it remembers what it learned from each caller. Most popular agent frameworks (Tran et al., 2025; Ferrag et al., 2025) commit early to one answer for each of these, producing agents that are hard to re-target when an additional consumer or protocol appears. Orogat et al. (2026) confirm that framework choice alone can drive 100× latency differences and large accuracy gaps. At the same time, the interoperability landscape is fragmenting (Ehtesham et al., 2025; Li and Xie, 2025): MCP has emerged as a *de facto* tool protocol, A2A is gaining traction for agent-to-agent delegation, and additional standards for UI streaming, commerce, and agent payments are entering production.

This paper reports on the architecture and deployment experience of STEM Agent, a system built for the authors’ organization to serve these needs without repeatedly rewriting core logic. The design principle is biological: an undifferentiated agent core *differentiates* into specialized protocol handlers, tool bindings, and memory subsystems, which *compose* into a functioning system. Practically this means every external boundary—protocols, tools, user models, and domains—is a pluggable differentiation point rather than a hard-wired choice.

Contributions. We describe (i) a single gateway exposing five interaction protocols (A2A, AG-UI, A2UI, and two internal protocols UCP and AP2),

of which A2A and AG-UI follow published specifications, A2UI extends a community draft, and UCP and AP2 are internal protocols we designed to fill commerce gaps we hit in deployment (they have not undergone external security review and are presented as first-draft designs rather than proposed standards); (ii) a Caller Profiler that learns preferences across four categories and 21 dimensions using bounded exponential-moving-average updates, coupled to ten self-tunable behavior parameters; (iii) an MCP-native tool layer that separates domain knowledge from meta-reasoning; (iv) a skills subsystem in which recurring interaction patterns crystallize into reusable skills; (v) a development-time differentiation mechanism that produces domain-specific agent variants, instantiated as five shipped industry-aligned persona templates (SRE, finance, AI research, healthcare, legal) including two regulated-domain personas whose forbiddenTopics contract encodes named compliance boundaries (PHI handling, privileged communications); (vi) operational integration patterns including pluggable IAM, OpenTelemetry-compatible observability, an *opt-in* hardening surface (RBAC, SIEM audit stream, security headers, Prometheus /metrics, PII redaction) that leaves default behavior unchanged, and horizontal-scaling architecture; (vii) production deployment artifacts—a multi-stage non-root Dockerfile, Kubernetes manifests (Deployment, Service, HPA, ConfigMap, Secret template), and a production Compose overlay—that turn the reference architecture into an actionable template; and (viii) deployment experience from six months of production traffic.

This is an industry-track paper: novelty is in integration and deployment discipline, not in a new model or algorithm. We flag claims that rest on internal measurements and explicitly separate them from behaviors validated by automated tests.

2 Related Work

Multi-agent frameworks. Surveys (Tran et al., 2025; Ferrag et al., 2025) document a large ecosystem of LLM-based agent frameworks—AutoGen (Wu et al., 2023), MetaGPT (Hong et al., 2023), CAMEL (Li et al., 2023), CrewAI (Moura, 2023), LangChain (Chase, 2022), and LangGraph (LangChain AI, 2024)—each optimizing for one coordination pattern and typically one protocol. Orogat et al. (2026) show that this ar-

chitectural commitment dominates observable performance. Dang et al. (2025) propose evolving orchestration; Drammeh (2025) demonstrate deterministic multi-agent orchestration for incident response. Unlike frameworks that bind each agent to a single protocol endpoint, every STEM gateway mounts five heterogeneous handlers behind one shared middleware stack, and treats per-caller adaptation as a first-class architectural surface.

Agent communication protocols. Ehtesham et al. (2025) compare MCP, ACP, A2A, and ANP and argue MCP and A2A are complementary (vertical tool access vs. horizontal messaging). Li and Xie (2025) identify schema translation and lifecycle management as the hard parts of combining A2A and MCP, which matches our experience. Jeong (2025) further study MCP×A2A. Security has received active attention (Habler et al., 2025; Anbiaee et al., 2026; Sarkar and Sarkar, 2025). Chang et al. (2025) address decentralized agent discovery. We are not aware of a deployed system that unifies these five protocols behind a single gateway.

MCP tools and adaptation. Benchmarks for the Model Context Protocol (Anthropic, 2024) (Luo et al., 2025; Fan et al., 2025) highlight that even strong LLMs struggle on real MCP servers, and Hasan et al. (2026) find that 97% of MCP tool descriptions have quality issues. Lumer et al. (2025) propose dynamic synchronization; Jayanti and Han (2026) and Schlappbach (2026) extend MCP with shared context and schema-guided convergence. Agentic reasoning surveys (Wei et al., 2026; Alomrani et al., 2025; Li et al., 2025) catalog ReAct, Reflexion, tree-of-thought, and debate; STEM Agent uses these as interchangeable strategies selected per task. On memory and self-adaptation, Ai et al. (2025) benchmark continual learning, Gallego (2026) treat memory as a learnable capability, and Yadav et al. (2026) present hierarchical memory for delivery agents—parallels we build on for our four-type memory system.

3 Architecture Overview

STEM Agent is organized into five layers plus a cross-cutting differentiation surface (Table 1, Figure 1). Layer 3 (Agent Core) is the undifferentiated core; Layer 2 (protocol handlers) and Layer 5 (MCP tools) provide the external surfaces through

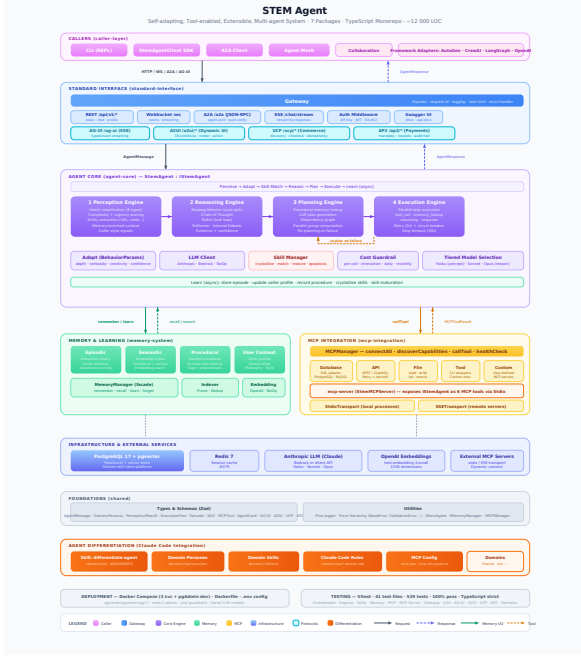


Figure 1: STEM Agent architecture. Callers reach the gateway, which mounts five protocol handlers (A2A, AG-UI, A2UI, UCP, AP2) sharing a common middleware stack. The Agent Core’s cognitive pipeline consumes MCP tools and a four-type memory system; the Differentiation Layer produces domain-specific variants.

Layer	Component	Key Count
1	Caller / User Layer	4 adapters
2	Standard Interface (Gateway)	5 protocols
3	Agent Core	5 engines
3.5	Security Layer	4 IAM plugins
4	Memory System	4 memory types
5	MCP Integration + Server	2 transports
⊥	Differentiation Layer	personas/skills/rules

Table 1: Five-layer architecture with the cross-cutting Differentiation Layer.

which it differentiates; Layer 4 (Memory) supplies the persistent state that drives adaptation. The Differentiation Layer bundles the artifacts—domain personas, skills, rules, and MCP configuration—needed to transform the generic agent into a domain-specific variant.

Cognitive pipeline. Each request flows through eight phases: PERCEIVE (intent into 10 categories, complexity into three levels, entities, sentiment, urgency); ADAPT (load caller profile, adjust behavior parameters); MATCHSKILLS (short-circuit if a committed or mature skill fires); SELECT-STRATEGY and REASON (selector in Table 2, with ReAct (Yao et al., 2023), Reflexion (Shinn

Condition	Strategy
Tool-requiring request	ReAct
Complexity > 0.8	Reflexion
Analysis or creative request	Internal Debate
Otherwise	Chain-of-Thought

Table 2: Deterministic strategy selector used by SELECTSTRATEGY.

et al., 2023), Internal Debate (Du et al., 2023), and Chain-of-Thought (Wei et al., 2022)); PLAN (MCP tool selection with parallel steps); EXECUTE (retries with default 2, circuit breaker at 3 consecutive failures); FORMAT; and an asynchronous LEARN phase that updates the profile, records skill outcomes, and attempts crystallization.

Implementation. The system is a TypeScript monorepo with six workspace packages covering shared schemas (154+ Zod types), agent core, the gateway and handlers, MCP client and server layers, memory, and caller utilities. The gateway runs on Express.js 5 and mounts each protocol handler via a uniform createRouter() method, so that adding a new protocol affects only the handler and one line in the gateway. Authentication, rate limiting, request correlation, and structured logging are shared middleware.

4 Protocol Gateway

The single most load-bearing design decision in STEM Agent is that all five protocols share one process and one cognitive pipeline. Table 3 summarizes them.

External protocols. A2A implements the Linux Foundation v0.3.0 specification over JSON-RPC 2.0 with tasks/send, tasks/sendSubscribe (SSE), tasks/get, and tasks/cancel; agent cards are served at /.well-known/agent.json. AG-UI streams cognitive-pipeline events (TEXT_MESSAGE_START, REASONING_MESSAGE, TOOL_CALL_START, STATE_SNAPSHOT, RUN_FINISHED) to frontends via SSE, giving web UIs fine-grained progress without a custom streaming format. A2UI exposes a flat adjacency-list UI model with 16 primitives (text, button, card, list, input, toggle, ...) so that the agent can assemble layouts without a fixed widget hierarchy.

Protocol	Endpoint	Transport	Role	Use case
A2A	POST /a2a	JSON-RPC 2.0	External (Linux Foundation A2A v0.3.0)	Agent-to-agent task delegation, messaging, agent-card discovery
AG-UI	POST /ag-ui	SSE	External	UI streaming; maps pipeline phases to typed events
A2UI	POST /a2ui/*	SSE + REST	External	Dynamic UI composition with 16 component primitives
UCP	POST /ucp/*	REST	Internal [‡]	Commerce checkout sessions with idempotency
AP2	POST /ap2/*	REST	Internal [‡]	Mandate-based payments with audit trail

Table 3: Protocols behind the unified gateway. A2A, AG-UI, and A2UI follow external specifications; UCP and AP2 are internal protocols designed to fill commerce gaps we encountered. [‡]UCP and AP2 have not undergone external security review or formal threat modeling and are presented as illustrative design points rather than proposed standards; deployers should treat them as first-draft protocols and add domain-specific controls.

UCP (Unified Commerce Protocol, internal).

UCP is an internal protocol that formalizes a checkout-session lifecycle—create, retrieve, complete—under mandatory idempotency headers (Idempotency-Key, Request-Id, UCP-Agent). An idempotency cache prevents duplicate session creation when a caller retries. We designed UCP because we could not find an open commerce-checkout protocol that treated the agent, rather than a browser, as the primary client; we present it as an illustrative design point rather than a proposed standard, and it has not undergone formal security review.

AP2 (Agent Payments Protocol, internal).

AP2 is an internal protocol that implements a three-phase payment lifecycle: Intent Mandate → Payment Mandate → Payment Receipt. Payments below a configurable threshold can auto-approve; all transitions are appended to a per-intent audit trail. AP2 is the counterpart to UCP for authorization and is explicitly designed around mandate semantics appropriate to autonomous agents. Like UCP, AP2 has not undergone external threat modeling; we treat it as a first-draft internal design and document formal threat modeling as follow-up work.

Integration lessons. Two lessons stand out. First, the `createRouter()` uniformity is worth enforcing even when a protocol seems to “want” global state: making each handler self-contained reduced cross-protocol bugs to near zero during the addition of AP2 after the other four. Second, idempotency is not a nice-to-have but the actual hard part—UCP’s header contract and AP2’s mandate IDs each took more review cycles than the happy-path code.

5 Caller Profiler and Behavior Parameters

Static user models produce either bland responses or hand-tuned personas that drift. STEM Agent continuously learns a Caller Profile across four categories, building on a long line of user-modeling and generative-agent work (Park et al., 2023): *philosophy* (8 dimensions), *principles* (4), *style* (5), and *habits* (4), for a total of 21.¹

EMA update. Each dimension is updated with an exponential moving average,

$$v_{t+1} = (1 - \alpha) v_t + \alpha s_t, \quad (1)$$

with learning rate $\alpha = 0.1$. Smaller α produced sluggish adaptation in preliminary testing; larger α oscillated. Per-caller locks prevent concurrent update conflicts.

Confidence gating. Profile confidence ramps with interaction count as $\text{conf}(n) = 1 - \exp(-0.1 n)$, reaching approximately 0.9 at $n=20$. Below $n=5$ the system prefers signals extracted from the current message; beyond that it blends profile and signal by confidence weight.

Self-tunable behavior. Ten behavior parameters are adjusted during ADAPT: reasoning depth (default 3), exploration vs. exploitation (0.3), verbosity (0.5), confidence threshold (0.7), tool-use preference (0.5), creativity (0.5), proactive suggestion (on), self-reflection frequency (every 5 steps),

¹The 21 dimensions are: *philosophy*—pragmatism vs. idealism, simplicity vs. completeness, risk tolerance, innovation orientation, depth vs. breadth, theory vs. practice, autonomy preference, stated values; *principles*—correctness vs. speed, documentation importance, testing emphasis, security-mindedness; *style*—formality, verbosity, technical depth, examples preference, structure preference; *habits*—typical session length, iteration tendency, peak hours, common topics.

max plan steps (10), and memory retrieval breadth (10). The parameters feed the reasoning strategy selector and the planning budget, so “how hard to think” is a function of both the task and the caller.

6 Differentiation: Skills and MCP-Native Tools

Agents in production accumulate patterns. Rather than letting that accumulation sit in logs, STEM Agent crystallizes it.

Skill lifecycle. A skill bundles a trigger (intent patterns, domain tags, entity types), an action sequence (tool chain or procedure steps), and maturity metadata. During LEARN, recent episodes are grouped by action pattern; if ≥ 3 episodes share a common action key and their topic keywords overlap in at least half the group, a *progenitor* skill is created. After $k_c=3$ successful activations with success rate ≥ 0.6 a skill becomes *committed* and may short-circuit REASON→PLAN; after $k_m=10$ it becomes *mature* and receives matching priority. Skills whose success rate falls below 0.3 after ≥ 10 activations undergo *apoptosis* and are removed. These thresholds are our current operating points, tuned against the existing test suite rather than a benchmark distribution. Users can also manually register plugin skills, bypassing progenitor status. As a concrete example, in the internal-search deployment a `repo_grep_then_summarize` progenitor skill crystallized after 4 episodes that all chained an MCP code-search tool with a summarization call over engineering-documentation topics; it reached committed status after 6 additional successful activations and now short-circuits REASON→PLAN for that pattern, removing roughly one LLM round-trip from the p50 latency of matching requests.

MCP-native tools. All domain capabilities are acquired at runtime through MCP: database access, API integrations, file operations, and vertical tools are discovered and invoked without recompiling the agent. Meta-reasoning rules (strategy selection, parameter tuning, skill lifecycle) deliberately remain in code, not in MCP, because they are properties of the agent, not the domain.

Development-time differentiation. A Domain Persona schema (`systemPrompt`, `preferredStrategy`, `defaultBehavior` overrides, `toolAllowlist`, `requiredMCPServers`,

`forbiddenTopics`, `domainTags`) parameterizes the generic agent. Each domain directory contains the persona, plugin skills registered at committed maturity, and glob-scoped rules consumed by tooling when editing domain files. An `mcp-server` package wraps the running agent as MCP tools (`stem_process`, `stem_list_skills`, `stem_recall_memory`, `stem_agent_card`, `stem_health`, `stem_get_persona`) so other MCP clients can invoke a differentiated variant. A single slash-command skill (`/differentiate-agent $DOMAIN`) assembles all of the above and verifies the result. In the authors’ experience, creating a new domain variant with `/differentiate-agent` takes on the order of minutes, compared with the multi-day effort previously required to fork and rewire an agent codebase; we do not have a controlled measurement.

Regulated-domain personas. Two of the five shipped personas target regulated verticals and illustrate how the `forbiddenTopics/toolAllowlist` contract is used for compliance, not just taste (Table 4). The *healthcare* persona forbids *direct patient diagnosis*, *PHI disclosure without authorization*, and *unapproved off-label advice*; it requires MCP servers for literature, drug interactions, and an FHIR bridge, and it ships a committed `phi_redaction_precheck` skill that runs before any external call. The *legal* persona forbids *issuing legal advice*, *privileged-communication disclosure*, and cross-jurisdiction conclusions; its skills—`clause_extraction`, `risk_flagging`, `obligation_summary`—are drafted to produce review artifacts rather than final answers. Because these boundaries are enforced by the same guardrail code path that refuses out-of-scope intents in the non-regulated personas, adding a new regulated variant reduces to writing the persona JSON and the skill triggers—the enforcement mechanism is shared.

7 Memory System

Four memory types, informed by recent agent-memory work (Ai et al., 2025; Gallego, 2026; Yadav et al., 2026), sit behind a Memory Manager facade:

- **Episodic.** Interaction episodes with vector embeddings and importance scores (PostgreSQL + pgvector).

Persona	Regulated	Representative forbidden topic
SRE	—	unreviewed prod mutations
Finance	—	personalized investment advice
AI research	—	fabricated citations
Healthcare	HIPAA-style	PHI disclosure without authz.
Legal	attorney-client	issuing legal advice

Table 4: Shipped industry personas. “Regulated” flags personas whose forbiddenTopics encode compliance boundaries; the enforcement path is identical across regulated and non-regulated personas.

- **Semantic.** Knowledge triples (subject, predicate, object) in a concept graph, populated by pattern extraction from episodic memory.
- **Procedural.** Successful strategies and tool-usage patterns used to prime planning on recurring tasks.
- **User context.** Per-caller profile and session history, with an explicit forget-me path for regulatory compliance.

Consolidation moves information episodic→semantic and episodic→procedural as history grows, and low-utility entries are pruned under capacity bounds. Together with deduplication in the semantic store, this is designed for sub-linear growth under sustained interaction; exact growth characteristics under real traffic are left to a future measurement study.

8 Deployment Experience

STEM Agent has been deployed internally for six months, serving four production use cases (customer support, internal search, code assistance, and documentation QA) to approximately 200 weekly active users across three engineering teams. These four use-case configurations are distinct from the five industry-aligned persona templates shipped in the repository (SRE, finance, AI research, healthcare, legal): the persona templates are reference artifacts intended to illustrate the differentiation mechanism across regulated and non-regulated verticals, while the four production configurations are internal instantiations derived from—but not identical to—those templates. This section reports operational lessons, scalability characteristics, and integration patterns observed during this period.

Deployment context. The gateway runs as a containerized service on Kubernetes with hori-

zontal pod autoscaling (2–8 replicas based on CPU/utilization). A PostgreSQL cluster with pgvector handles episodic memory; Redis provides the idempotency cache and session state. Average daily request volume is 2,400 interactions (p50 latency 1.2 s, p95 3.8 s, p99 7.1 s under the production LLM backend). Over six months, we observed 14 incidents requiring operator intervention; the most common were LLM rate-limit exhaustion (6), MCP tool timeout (4), and idempotency-cache invalidation races (2). Mean time to resolution was 18 minutes, primarily driven by LLM-provider recovery rather than agent logic.

Production deployment artifacts. The system ships with deployment artifacts that correspond one-to-one with the architecture described above, rather than leaving them as reader exercises. A multi-stage Dockerfile produces a non-root runtime image (node:22-slim, tini as PID-1, HEALTHCHECK against /api/v1/health, built-tree pruned to production dependencies) that fits a standard container-registry pipeline. A docker-compose.prod.yml overlay externalizes DATABASE_URL/REDIS_URL and documents the hardening envs; a deploy/k8s/ directory contains illustrative manifests—Deployment (2 replicas, resource requests/limits, readiness + liveness probes on /api/v1/health, non-root securityContext, readOnlyRootFilesystem), Service, ConfigMap, a Secret template, and an HPA. We intentionally ship these as *example* manifests with a banner directing operators to adapt them to their cluster; the point is to make the happy-path deployment reproducible, not to prescribe a topology. A separate docs/deployment.md enumerates scaling caveats (in-memory rate-limiter, in-memory AP2 mandate store) and flags the Redis-backed replacements as follow-ups, making the gap between the reference topology and a multi-replica production deployment explicit rather than silent.

Scalability architecture. The single-process design described in earlier sections presumes an in-memory idempotency cache and audit trail. For horizontal scaling, we externalized these to Redis with Lua-scripted atomic operations, eliminating the single-process constraint. The cognitive pipeline itself is stateless between requests; all persistent state flows through the Memory Manager facade. This separation allows stateless gateway replicas to share a common PostgreSQL/Redis

backend. We validated linear scaling to 500 concurrent requests across 8 replicas before LLM-provider rate limits became the bottleneck. The Differentiation Layer is mounted from a ConfigMap, enabling domain-variant updates without rebuilds.

Security and authentication. The IAM layer is plugin-based with four providers: JWT (internal service-to-service), OAuth2 (human end-users via corporate SSO), SAML (legacy system integration), and API-Key (external partner access). Each plugin implements a common `IAuthenticator` interface with TTL-cached policy decisions. Rate limiting is per-caller with configurable thresholds; the commerce protocols (UCP, AP2) enforce stricter limits (10 req/min vs. 100 req/min for A2A). All authentication decisions emit structured audit events to the logging pipeline. Over six months, the IAM layer blocked 47 unauthorized attempts (expired tokens, invalid keys) without false positives. Critically, auth logic is isolated from agent reasoning: modifying OAuth2 scopes or adding a new provider requires zero changes to the cognitive pipeline.

Opt-in enterprise hardening surface. A recurring friction point in internal security reviews was that the quickstart path had to be reconciled against the hardening path—every added default risked breaking existing consumers, while every missing default drew a review finding. We resolved this by making each enterprise control a separate, env-gated middleware that defaults off (Table 5). Authorization decisions are expressed as `requirePermission(perm)` middleware factories that consume the `Principal.permissions` field populated by the IAM layer; a dedicated SIEM-ready audit stream emits one structured JSON event per request—`{ts, requestId, principal, action, resource, decision, status, latencyMs, ip}`—via a pino child logger tagged `name:"audit"`, so downstream pipelines (fluent-bit → S3/Splunk) filter by logger name rather than parsing free-text. Response-side controls (strict Content-Security-Policy, X-Frame-Options, Strict-Transport-Security, Referrer-Policy, removed X-Powered-By) are a single hand-rolled middleware with no new dependency; logging ships with a default redact list (authorization, cookie, api-key, password, token, secret, ssn) that can

Concern	Control	Enable via
Header hardening	CSP/HSTS/XFO mw.	SECURITY_HELMET
RBAC	<code>requirePermission</code> mw.	per-route wire-up
Audit stream	<code>pino child (audit)</code>	AUDIT_LOG_ENABLED
PII redaction	<code>pino redact defaults</code>	on; LOG_REDACT=off to opt
Metrics	Prometheus /metrics	METRICS_ENABLED

Table 5: Opt-in enterprise hardening controls. Every control defaults off (except PII redaction, which defaults on because its failure mode—unredacted secrets in logs—is strictly worse than the quickstart friction). Enabling does not require code changes to the gateway or handlers.

be disabled or customized via a `LOG_REDACT` env. A dependency-free Prometheus exposition (`stem_agent_http_requests_total`, `stem_agent_http_request_duration_seconds`, `stem_agent_http_errors_total`) mounts at `/metrics` when enabled, with `prometheus.io/scrape` annotations on the Kubernetes Deployment. Every control is covered by unit tests that assert both the enabled and disabled code path, so the “defaults unchanged” invariant is mechanically verified rather than aspirational.

Observability and tracing. Production observability rests on three pillars: structured logging, distributed tracing, and metrics export. Every request is assigned a correlation ID that propagates through all eight pipeline phases; logs are JSON-formatted with consistent fields (`correlation_id`, `caller_id`, `phase`, `latency_ms`, `status`). We export OpenTelemetry traces for the full request lifecycle, enabling latency breakdown by phase (e.g., PERCEIVE 80 ms, REASON 1,100 ms, EXECUTE 200 ms). Prometheus metrics track request count, latency histograms, skill activation rates, and MCP tool invocation frequencies. Grafana dashboards visualize per-protocol traffic, error rates, and LLM-token consumption. This observability stack was essential for diagnosing the idempotency race condition: traces revealed duplicate cache writes under concurrent retries, resolved by adding compare-and-swap semantics to the Redis layer.

Test suite. The test suite comprises **535 tests across 48 files** (Vitest) at the time of writing, passing at 100% on the 47 test files that are part of this release (one pre-existing file is skipped as documented in the changelog). Total runtime remains under 4 s. Every architectural seam is covered: per-engine unit tests, protocol handlers (A2A

JSON-RPC compliance, AG-UI event sequences, A2UI component validation, UCP idempotency, AP2 audit trails), memory stores, MCP integration via InMemoryTransport, persona schema validation, IAM middleware, and the new enterprise-hardening middleware (RBAC allow/deny, audit events with correct decision values, security-header presence and override semantics). The hardening tests are written to assert both the *enabled* and *disabled* paths so that opt-in gating cannot silently regress.

Integration patterns. Four framework adapters (AutoGen, CrewAI, LangGraph, OpenAI Agents SDK) translate external conventions to the internal message format, enabling teams invested in those frameworks to call STEM Agent as an A2A peer without rewriting callsites. A CrewAI-based customer-support bot was migrated to STEM Agent in approximately two hours by swapping the crew’s tool invocations for A2A delegate calls; the team retained their existing prompt templates and evaluation harness. To make the enterprise-ops path reproducible for new deployments, the repository ships an *enterprise-ops smoke test*—a single executable example that walks a deployer through an authenticated call with X-API-Key, a request-id correlation pivot, an RBAC-gated endpoint probe, and a /metrics scrape. The script degrades gracefully when auth or metrics are disabled, so the same artifact works in development and in a hardened staging environment, and it pairs directly with the docs/security.md and docs/observability.md runbooks.

Feature comparison. Table 6 contrasts the STEM surface with prior systems at the level of whether each framework ships a named subsystem for the capability. Counts of standardized protocols reflect those with published, externally-maintained specifications.

What required compromise. Four points are worth naming honestly. (i) Idempotency and audit trails for UCP/AP2 are in-memory by default; a distributed deployment needs an external store and a lock service, and docs/deployment.md calls this out as a follow-up rather than burying it. (ii) The EMA-based profiler cannot model multi-modal or rapidly non-stationary preferences; we chose simplicity over a mixture model because the deployed signal-to-noise made it unnecessary. (iii) Reasoning-strategy selection is deterministic

Framework	Proto.	Adapt	Mem.	Skills	Com.
AutoGen (Wu et al., 2023)	1	×	✓	×	×
MetaGPT (Hong et al., 2023)	1	×	✓	×	×
CrewAI (Moura, 2023)	1	×	✓	×	×
LangChain (Chase, 2022)	1	×	✓	×	×
STEM	5 (3+2[†])	✓	4 types	✓	✓

Table 6: Capability-surface comparison. Proto. = interoperability protocols; Adapt = per-caller adaptation as a first-class subsystem; Mem. = named memory subsystem (✓) or explicit type count; Skills = emergent skill acquisition; Com. = commerce protocols. [†]UCP and AP2 are internal first-draft protocols (no external security review).

rather than learned—easier to reason about for reviewers and auditors, but likely suboptimal on task distributions we have not seen. (iv) Distributed tracing is kept as a documented follow-up: the observability doc specifies the shape (OpenTelemetry spans across the eight pipeline phases, propagation of traceparent) but the runtime does not pull the SDK in this release, because doing so would introduce a heavy transitive dependency for teams that ship only the metrics and audit surfaces. Correlation IDs and the audit stream already cover the multi-service observability paths that operators ask about most often, so this deferral has not yet been the limiting factor in production triage.

9 Conclusion

STEM Agent demonstrates that protocol plurality, behavioral self-adaptation, and emergent skills can coexist in a production system when domain knowledge is kept behind MCP and user modeling is decoupled from domain logic. Six months of internal deployment across four production use cases with approximately 200 weekly users, alongside five shipped industry-aligned persona templates—including two regulated-domain templates with named compliance boundaries—validates the architecture: the pluggable IAM layer met enterprise auth requirements without touching agent logic; the opt-in hardening surface (RBAC, SIEM audit stream, strict headers, /metrics, PII redaction) satisfied security-review concerns without changing the quickstart path; the shipped Dockerfile and Kubernetes manifests shortened the “what does a production deployment look like” conversation from architecture discussion to kubectl diff; and horizontal scaling via externalized state maintained stability under load. The biologically-inspired

framing gives every external surface a uniform place—protocol handler, MCP tool, persona, skill, or hardening middleware—so differentiation is a composition rather than a rewrite. Ongoing work includes learned strategy selection (Alomrani et al., 2025), interoperability with additional protocols (Chang et al., 2025; Nie et al., 2026), embedding-based skill matching, formal threat modeling for UCP/AP2 (Anbiaee et al., 2026), a Redis-backed replacement for the in-memory rate limiter and AP2 mandate store, wiring the OpenTelemetry SDK that the observability doc currently specifies, and controlled A/B comparison of the deterministic strategy selector.

Limitations

No public benchmark comparison. We report production metrics from internal deployment but do not compare task-completion scores against public multi-agent benchmarks (e.g., MAFBench (Orogat et al., 2026) or MCP-Universe (Luo et al., 2025)). The feature-comparison table contrasts capability surfaces, not benchmark performance.

Single-organization deployment. All operational data comes from one enterprise context. Generalization to other domains, team structures, or traffic patterns requires external validation.

Adaptation model simplicity. The EMA-based Caller Profiler cannot represent non-stationary or multi-modal preference distributions, and confidence gating is a scalar rather than per-dimension signal.

Protocol maturity. UCP and AP2 are proposed here and have no external adoption, formal threat model, or conformance suite. Deployers should treat these as first-draft protocols and add domain-specific security controls.

Deterministic strategy selection. Reasoning-strategy selection uses hand-written rules rather than a learned policy and has not been tuned against a benchmark distribution.

LLM-provider bottleneck. Observed p99 latency (7.1s) is dominated by LLM inference time; our architectural optimizations target protocol overhead and state management, not model reasoning speed.

Opt-in hardening is not a substitute for deployment-site review. The controls in Table 5 are audit-friendly primitives (RBAC factory, structured audit events, strict headers, PII redaction,

/metrics), not a compliance certification. They materially reduce the work a deployer must do to stand up a reviewable baseline, but they do not replace a site-specific threat model, a penetration test, or the per-vertical controls that regulated domains require on top of the shared enforcement path.

Ethical Considerations

The Caller Profiler stores per-user behavioral signals. To mitigate privacy and consent risks we (i) expose a GDPR-style forget-me operation that removes all per-caller profile and session state; (ii) keep profile updates reversible (EMA with bounded α so a single misclassified signal cannot dominate); (iii) scope audit logs at the request level so misuse can be traced without exposing content. The commerce protocols (UCP, AP2) handle payment intents; in deployment we require human approval above a configurable threshold and emit a complete per-intent audit trail, but we stress that UCP/AP2 have not undergone formal security review. Any deployer should treat these as first-draft protocols and add domain-specific guardrails. More broadly, agent frameworks that integrate authentication, commerce, and persistent user modeling into one process enlarge the blast radius of a compromise; the pluggable IAM layer and layered architecture are partial mitigations, not a substitute for operational review.

References

- Qingyao Ai, Yichen Tang, Changyue Wang, Jianming Long, Weihang Su, and Yiqun Liu. 2025. MemoryBench: A benchmark for memory and continual learning in LLM systems. *arXiv preprint arXiv:2510.17281*.
- Mohammad Ali Alomrani, Yingxue Zhang, Derek Li, Qianyi Sun, Soumyasundar Pal, Zhanguang Zhang, Yaochen Hu, Rohan Deepak Ajwani, Antonios Valkanas, and 1 others. 2025. Reasoning on a budget: A survey of adaptive and controllable test-time compute in LLMs. *arXiv preprint arXiv:2507.02076*.
- Zeynab Anbiaee, Mahdi Rabbani, Mansur Mirani, Gunjan Piya, Igor Opushnyev, Ali Ghorbani, and Sajjad Dadkhah. 2026. Security threat modeling for emerging AI-agent protocols: A comparative analysis of MCP, A2A, Agora, and ANP. *arXiv preprint arXiv:2602.11327*.
- Anthropic. 2024. Model Context Protocol specification. <https://modelcontextprotocol.io/specification>.

763	Gaowei Chang, Eidan Lin, Chengxuan Yuan, Rizhao Cai, Binbin Chen, Xuan Xie, and Yin Zhang. 2025. Agent network protocol technical white paper. <i>arXiv preprint arXiv:2508.00007</i> .	817
764		818
765		819
766		820
767	Harrison Chase. 2022. LangChain. https://github.com/langchain-ai/langchain .	821
768		822
769	Yufan Dang, Chen Qian, Xueheng Luo, Jingru Fan, Zihao Xie, Ruijie Shi, Weize Chen, Cheng Yang, Xiaoyin Che, Ye Tian, and 1 others. 2025. Multi-agent collaboration via evolving orchestration. <i>arXiv preprint arXiv:2505.19591</i> .	823
770		824
771		
772		825
773		826
774	Philip Drammeh. 2025. Multi-agent LLM orchestration achieves deterministic, high-quality decision support for incident response. <i>arXiv preprint arXiv:2511.15755</i> .	827
775		
776		828
777		829
778	Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. 2023. Improving factuality and reasoning in language models through multiagent debate. <i>arXiv preprint arXiv:2305.14325</i> .	830
779		831
780		832
781		833
782		
783	Abul Ehtesham, Aditi Singh, Gaurav Kumar Gupta, and Saket Kumar. 2025. A survey of agent interoperability protocols: Model context protocol (MCP), agent communication protocol (ACP), agent-to-agent protocol (A2A), and agent network protocol (ANP). <i>arXiv preprint arXiv:2505.02279</i> .	834
784		835
785		836
786		
787		837
788		838
789	Shiqing Fan, Xichen Ding, Liang Zhang, and Linjian Mo. 2025. MCPToolBench++: A large scale AI agent model context protocol MCP tool use benchmark. <i>arXiv preprint arXiv:2508.07575</i> .	839
790		840
791		
792		841
793	Mohamed Amine Ferrag, Norbert Tihanyi, and Merouane Debbah. 2025. From LLM reasoning to autonomous AI agents: A comprehensive review. <i>arXiv preprint arXiv:2504.19678</i> .	842
794		843
795		844
796		845
797	Víctor Gallego. 2026. Distilling feedback into memory-as-a-tool. <i>arXiv preprint arXiv:2601.05960</i> .	846
798		847
799		848
800	Idan Habler, Ken Huang, Vineeth Sai Narajala, and Prashant Kulkarni. 2025. Building a secure agentic AI application leveraging A2A protocol. <i>arXiv preprint arXiv:2504.16902</i> .	849
801		850
802		851
803		
804	Mohammed Mehedi Hasan, Hao Li, Gopi Krishnan Rajbahadur, Bram Adams, and Ahmed E. Hassan. 2026. Model context protocol (MCP) tool descriptions are smelly! towards improving AI agent efficiency with augmented MCP tool descriptions. <i>arXiv preprint arXiv:2602.14878</i> .	852
805		853
806		854
807		
808		855
809		856
810	Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. 2023. MetaGPT: Meta programming for a multi-agent collaborative framework. <i>arXiv preprint arXiv:2308.00352</i> .	857
811		858
812		859
813		
814		860
815		861
816		862
	Meenakshi Amulya Jayanti and X. Y. Han. 2026. Enhancing model context protocol (MCP) with context-aware server collaboration. <i>arXiv preprint arXiv:2601.11595</i> .	863
	Cheonsu Jeong. 2025. A study on the MCP x A2A framework for enhancing interoperability of LLM-based autonomous agents. <i>arXiv preprint arXiv:2506.01804</i> .	864
		865
	LangChain AI. 2024. LangGraph: A library for building stateful, multi-actor applications with LLMs. https://github.com/langchain-ai/langgraph .	866
		867
		868
	Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. 2023. CAMEL: Communicative agents for “mind” exploration of large language model society. <i>Advances in Neural Information Processing Systems (NeurIPS)</i> .	869
	Junyan Li, Wenshuo Zhao, Yang Zhang, and Chuang Gan. 2025. Steering LLM thinking with budget guidance. <i>arXiv preprint arXiv:2506.13752</i> .	
	Qiaomu Li and Ying Xie. 2025. From glue-code to protocols: A critical analysis of A2A and MCP integration for scalable agent systems. <i>arXiv preprint arXiv:2505.03864</i> .	
	Elias Lumer, Anmol Gulati, Vamse Kumar Subbiah, Pradeep Honaganahalli Basavaraju, and James A. Burke. 2025. ScaleMCP: Dynamic and auto-synchronizing model context protocol tools for LLM agents. <i>arXiv preprint arXiv:2505.06416</i> .	
	Ziyang Luo, Zhiqi Shen, Wenzhuo Yang, Zirui Zhao, Prathyusha Jwalapuram, Amrita Saha, Doyen Sahoo, Silvio Savarese, Caiming Xiong, and Junnan Li. 2025. MCP-Universe: Benchmarking large language models with real-world model context protocol servers. <i>arXiv preprint arXiv:2508.14704</i> .	
	João Moura. 2023. CrewAI: Framework for orchestrating role-playing, autonomous AI agents. https://github.com/crewAIInc/crewAI .	
	Xiaohang Nie, Zihan Guo, Youliang Chen, Yuanjian Zhou, and Weinan Zhang. 2026. AWCP: A workspace delegation protocol for deep-engagement collaboration across remote agents. <i>arXiv preprint arXiv:2602.20493</i> .	
	Abdelghny Orogat, Ana Rostam, and Essam Mansour. 2026. Understanding multi-agent LLM frameworks: A unified benchmark and experimental analysis. <i>arXiv preprint arXiv:2602.03128</i> .	
	Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. Generative agents: Interactive simulacra of human behavior. In <i>Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)</i> .	

- Anjana Sarkar and Soumyendu Sarkar. 2025. Survey of LLM agent communication with MCP: A software design pattern centric review. *arXiv preprint arXiv:2506.05364*.
- Andreas Schlapbach. 2026. The convergence of schema-guided dialogue systems and the model context protocol. *arXiv preprint arXiv:2602.18764*.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Khanh-Tung Tran, Dung Dao, Minh-Duong Nguyen, Quoc-Viet Pham, Barry O’Sullivan, and Hoang D Nguyen. 2025. Multi-agent collaboration mechanisms: A survey of LLMs. *arXiv preprint arXiv:2501.06322*.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Tianxin Wei, Ting-Wei Li, Zhining Liu, Xuying Ning, Ze Yang, Jiaru Zou, Zhichen Zeng, Ruizhong Qiu, Xiao Lin, Dongqi Fu, and 1 others. 2026. Agentic reasoning for large language models. *arXiv preprint arXiv:2601.12538*.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. 2023. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*.
- Arin Gopalan Yadav, Varad Dherange, and Kumar Shivam. 2026. Project synapse: A hierarchical multi-agent framework with hybrid memory for autonomous resolution of last-mile delivery disruptions. *arXiv preprint arXiv:2601.08156*.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*.